BOOSTK DOCUMENTATION

May 2026

Table of Contents

Dev Environment Setup Guide

This guide summarizes the steps to set up and run the Boostk project locally.

Installations:

Prerequisites

- Bun installed (Version 1.1+ latest recommended).
- Access to a PostgreSQL database (e.g., Supabase).
- Git installed for source control.

Install Bun (Windows)

Run the following command in a PowerShell terminal While not strictly required, Administrator access is recommended to ensure environment variables are updated correctly:

```
powershell -c "irm bun.sh/install.ps1 | iex"
```

Note: Restart your terminal after installation to ensure the *bun* command is recognized. You can verify the installation by running:

```
bun -v
```

Getting the Sources

Clone the repository locally using Git:

```
git clone https://github.com/FORHU/boostk-app.git
cd boostk-app
```

---

1. Initial Setup

- Frontend & Core

  Navigate to the root directory and install dependencies:

```
bun install
```

- Database Configuration

  Create a '.env' file in the root directory (boostk-app/) and populate it with the following configuration (replace DATABASE_URL with your own from Supabase):

```
VITE_SOCKET_URL=http://localhost:3001

DATABASE_URL=postgresql://postgres.[YOUR_PROJECT_REF]:[YOUR_PASSWORD]@aws-1-[REGION].pooler.supabase.com:5432/postgres

BETTER_AUTH_SECRET=ucLdSvBeaIhq6KYZnH3KL5f9Gj5Cvgh0
BETTER_AUTH_URL=http://localhost:3000/

RABBITMQ_URL=amqp://FXxbaGM1k5e9pesl:bCA60RzYK0hXthOKYrxX2eR28
```

```
ovV8U6s@158.178.241.113:5672

OPENAI_API_KEY=sk-proj-8bwXGGrF1I-jy5H4jATYpU2GGsEt5sHthvdCvI1
MKtvuh32ryaS1o4A22rWpROYvOEdYYJaTiET3BlbkFJ2AEUH9QxDIjU-vyQXRP
eJ7m9s1uOJg0gTf0CobLP4-LWeSkxxIpaQ4QbiUmGlCoYOwnPxutc0A
```

- Initialize Database
  For first-time implementation, initialize the Prisma client, run migrations, and seed the database using the consolidated script:

```
bun run setup-local-db
```

CAUTION

Warning: Running this command includes "prisma migrate reset" which will completely reset your database and erase all existing data. Only use this for initial setup or when you explicitly need a clean state.

Alternative Manual Database Setup(Optional):

If you prefer to run the steps individually or need more control:

1. Generate Prisma Client:

```
bun prisma generate
```

2. Reset/Apply Migrations: (Note: This will erase existing data)

```
bun prisma migrate reset
```

3. Seed the Database

```
bun prisma db seed
```

2. Running the Project

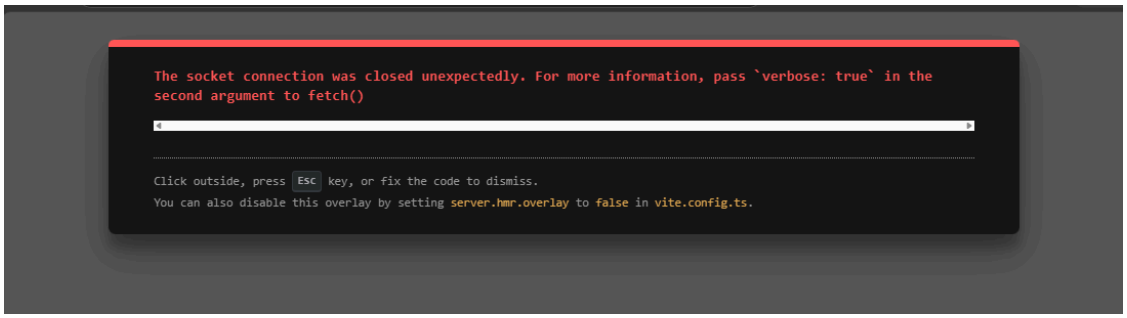
- Terminal: Run the development server from the boostk-app root:

```
bun run dev
```

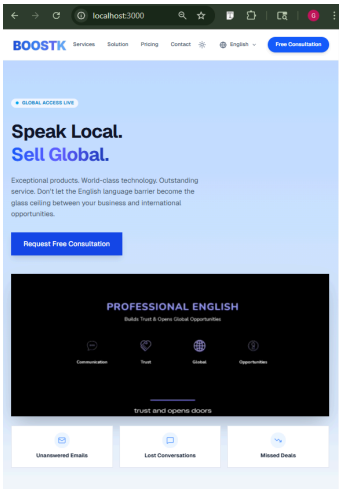
Accessible at: http://localhost:3000

3. Accessing the App

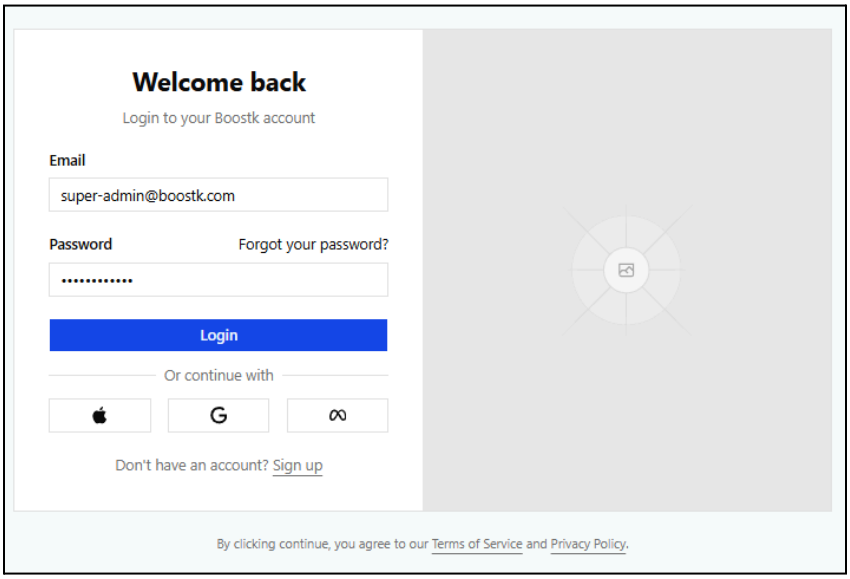
Note: Refresh the page when this error appears



- Open your browser and navigate to dashboard: http://localhost:3000



- Open your browser and navigate to admin login : http://localhost:3000/signin



4. Login Credentials (Seed Data)

- If you used the seed script, you can log in with:

```
Email: super-admin@boostk.com
Password: password123!
```

NOTE: Additional test accounts (support, agents, staff, etc.) are defined in prisma/seeds/user.seeder.ts. All accounts currently route to the default admin dashboard, as separate role-based routing has not yet been implemented.

Flow in the Code

This document outlines the architecture and directory structure of the BOOSTK project to help developers understand how the code is organized and how to navigate it.

Important Root Files:

- *README.md*: Primary setup and execution instructions.
- *.env*: Local environment variables (Database URLs, API secrets). Essential for database seeding.
- *package.json*: Project dependencies and automation. Note: Bun is the mandatory runtime.
    - *bun run dev*: Starts the application.
    - *bun run setup-local-db*: Initializes the database environment.
    - *bun run format*: Fixes code formatting using Biome.
    - *bun run lint*: Checks for code quality issues using Biome.
- *biome.json*: Configuration for Biome (Linting & Formatting).

Directory Structure & Flow

1. **src/** (Main Application Logic)
   The frontend is built with TanStack Start.
    - **routes/**: Page structure and navigation logic.
    - **components/**: Reusable UI components.
    - **lib/**: Shared utilities (DB client, Auth configuration).
    - **modules/**: Feature-specific logic (Chat, Organization management, etc.).
    - **styles.css**: Tailwind CSS entry point.
2. **prisma/** (Database Layer)
    - **schema.prisma**: The source of truth for the DB schema.
    - **seed.ts**: Script to populate the database with initial data for development.
3. **hono/** (Real-time Service)
    - Role: This is a dedicated Socket.io server running on port 3001.

- Usage: It handles all real-time events such as chat messages (new_message), room management (join_room), and ticket updates (ticket_list_updated).

4. **public/** & **docs/**
    - Static assets and additional technical documentation.

Development Standards & Rules

To keep the project maintainable and clean, all developers must follow these rules:

1. **Code Quality (Biome)**
    - We use **Biome** for both linting and formatting.
    - **Auto-Formatting:** Biome is configured to automatically format and fix basic linting issues whenever you **save** a file.
    - **Manual Formatting:** Use bun run format for bulk changes, if you are using a different editor, or to verify the entire project's consistency before committing.
    - **Linting**: Run bun run lint to check for logical errors, security issues, and complex styling problems that formatting cannot catch.
        - **How to fix errors:** Many issues are fixed automatically on save. For errors that remain, Biome will provide a clear explanation and often suggest the exact code change needed. You must resolve these manually by following the CLI suggestions before the code is considered "clean."
2. **Naming Conventions**
    - Folders: Must be all lowercase (e.g., src/modules/chat-service).
    - Files: Must follow kebab-case (e.g., ticket-details.tsx, user-profile.tsx).
3. **Module Pattern**
    - When adding a new feature, create a dedicated folder in src/modules/ using the lowercase naming rule.
    - Keep logic focused and modular to prevent the src/routes/ directory from becoming too complex.
4. **Runtime**
    - **Bun only:** To keep our development fast and consistent, we use Bun for all tasks. Please use bun for running scripts and managing packages instead of npm, yarn, or pnpm.

WEBSITE FLOW

May 2026

BoostK System Overview

BoostK is a dual-purpose platform combining a real-time agent-customer chat support system with a global business services hub. The chat system enables Korean businesses to manage customer conversations efficiently through role-based agent dashboards, while the homepage layer promotes BoostK's core mission: connecting Korean and Japanese SMEs with English-speaking professionals in the Philippines to bridge global market entry barriers.

This document provides a comprehensive overview of the BoostK platform, analyzing the current state of the codebase against the project's strategic goals.

1. Technical Architecture Summary

BoostK is built as a high-performance, real-time application using a modern unified stack.

Framework: TanStack Start (Full-stack React with SSR and Server Functions).
Runtime: Bun for high-speed execution and package management.
Database: PostgreSQL with Prisma ORM.
Real-time: Socket.io running on a standalone Hono server (port 3001).
Authentication: Better Auth with RBAC and Organization plugins.
AI Integration: OpenAI SDK for real-time multilingual translation.
Styling: Tailwind CSS 4.0 + Shadcn UI components.

2. Folder Structure Analysis

Directory	Responsibility	Key Files / Subdirectories
src/routes	File-based routing & SSR loaders	(app), (auth), (public), api
src/modules	Core business logic (Server Functions)	auth, organization, project, ticket, ticket-message
src/components	Reusable UI components	chat-support, landing-page, ui, organization

src/lib	Shared utilities & service clients	auth.ts, prisma.ts, socket.ts, openai.ts
hono/	Real-time WebSocket server	src/index.ts (Socket.io handlers)
prisma/	Database schema & migrations	schema.prisma, seeds/

3. Feature Implementation AuditPages

Feature	Branch Status	Implementation Level	Findings
RBAC / Auth	✅	Full Integration	better-auth is fully integrated with detailed role-based route protection.
AI Translation	✅	Integrated	Bidirectional OpenAI translation logic is active in message processing functions.
Homepage	✅	Integrated	High-fidelity landing page with multiple sections (Hero, Global Impact, Barriers, etc.).
Archiving	✅	Functional	Support for ending conversations and moving them to a dedicated "Archived Chats" record.

Frontend-Only / Mock-Reliant Features
These features have high-fidelity UI but currently depend on mock data or lack full backend integration.

Feature	UI Fidelity	Backend Status	Findings
Dashboard (Hub)	High	Mock-Heavy	Live metrics and conversation lists currently use static mock data for demo purposes.

Chat Support	High	Partial Mock	Core messaging is live, but attachments and status updates (Read/Failed) are simulated.
Admin / Users	High	Not Linked	Interface for managing global users and roles is designed but not connected to API.
Admin / Orgs	High	Not Linked	Organization management and role assignment UI exists as placeholders.
Admin / Settings	High	Not Linked	System-wide settings and configuration panels are UI-ready but non-functional.

4. Key Implementation Details

Real-Time & Multilingual Logic

The system uses a unique "Translation-as-a-Service" flow:

1. **Customer Message:** Detects language -> Translates to English (Agent's language) -> Stores both.
2. **Agent Message:** Identifies customer's language -> Translates English to Customer Lang -> Stores both.
3. **WebSocket:** Emits new_message event via Socket.io; clients invalidate TanStack Query keys for instant updates.

Data Model (Prisma)

The schema is highly relational and supports advanced features:

Organization 1:N **Project**

Project 1:N **Ticket**

Ticket 1:N **TicketMessage**

OrganizationRoleTemplate: Provides standardized role sets for new organizations.

5. Proposed Roadmap & Next Steps

Phase 1: Production Readiness (High Priority)

- **Backlinking Frontend to Backend:** Connect the high-fidelity Dashboard and Chat Support UI to live Prisma queries and Socket.io events, replacing all mockConversations and MOCK_MESSAGES.
- **Authentication Finalization:** Ensure the better-auth admin plugin is fully configured for user management.
- **Notification Engine:** Finalize the src/lib/notifier to provide real-time audio and desktop alerts.

Phase 2: User Experience (Medium Priority)

- **Profile Management:** Implement the interface for agents to update their personal and department info.
- **Customer Management Panel:** Enhance the "Customer Details" sidebar into a full CRM-style view.
- **Mobile Optimization:** Ensure the "Command Center" remains efficient on smaller screens.

Phase 3: Advanced Intelligence (Future)

- **AI Auto-Replies:** Suggest template responses based on the context of the customer's question.
- **Sentiment Tracking:** Add visual indicators for customer sentiment based on chat history.
- **Analytics Export:** Add CSV/PDF export for performance reporting and billing.

6. Technical Insights

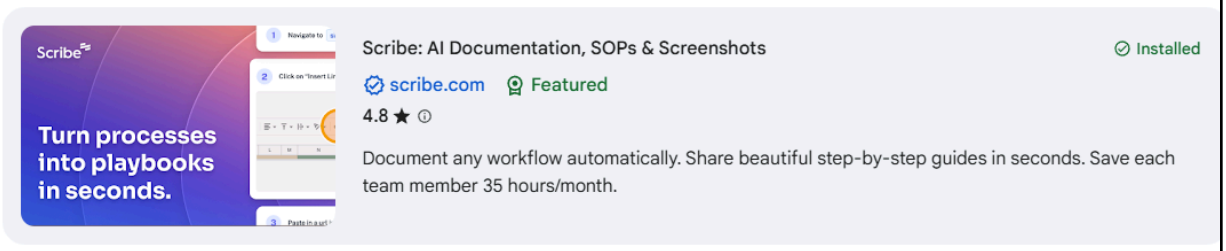
- **Unified Stack:** The app uses TanStack Start for a seamless full-stack experience
- **Hono Role:** Hono serves as the dedicated WebSocket orchestrator, ensuring low-latency message delivery.

Web Pages

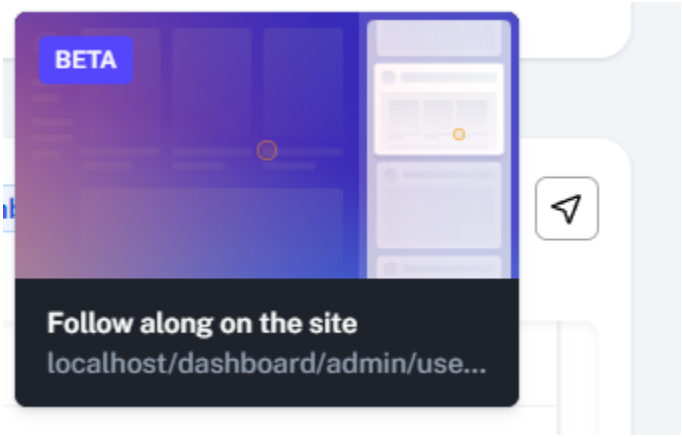
Links:

Figma - Pages Drafting:
https://www.figma.com/design/Xhu8ClvYe8kPYZvsfUIO2j/Forhu-BoostK-?node-id=0-1&p=f&t=hqpkLILi16TOXW22-0

Note: Add Scribe from google extension to access the walkthrough feature



Open link and hover the right corner of step 2 to follow along



Scribe - Walkthrough Dashboard:
https://scribehow.com/viewer/Managing_Customer_Support_Chats_and_Templates__5UoVnbGcRYy0h0dPxpO2iA

Scribe - Admin Control:
https://scribehow.com/viewer/Managing_Users_and_Organizations_in_the_Admin_Dashboard__b-yu7IhaS-CCyd_67y7kkg

7. Detailed Feature Specifications

A. Users & Role-Based Access Control

Status: Partially Implemented

BoostK implements a role-based system ensuring secure, scoped access across the platform. All routes are protected based on these role assignments, ensuring users only access features appropriate to their permissions.

TODO: Role routing per page, own page for roles

B. Dashboard (Project Overview)

Status: Frontend Implemented (Frontend High-Fidelity)
The Dashboard serves as the command center, providing real-time operational intelligence. It displays live counts of active conversations, average response times, and a scrollable list of active conversations with search and filter functionality.

TODO: Backend Functionality

A. Overview of

I. Active chats count

- Display total number of active conversations
- Real-time count updates
- Click to filter active chats

II. Average response time

- Calculate average time from customer message to agent reply

III. List of active conversations with preview and timestamps

- List view layout
- Customer name display
- Customer email display
- Last message snippet preview (first 50-100 characters)
- Timestamp of last activity (relative time: "2 min ago" or absolute)

B. Search and filter Search and filter chats (by agent, customer, department, or keyword)

- Search bar for conversations
- Filter by assigned agent
- Filter by department

C. Sorting options

- Sort by most recent activity
- Sort by unread messages first
- Toggle sort order

B. Search and filter Search and filter chats (by agent, customer, department, or keyword)

- Search bar for conversations
- Filter by assigned agent
- Filter by department

C. Sorting options

- Sort by most recent activity
- Sort by unread messages first
- Toggle sort order

C. Chat System

Status: Partially Implemented
(Core Features: Implemented | Transfer Feature: Not implemented) Organized into a three-column layout (Conversation List, Active Chat, Customer Details). Features real-time bidirectional messaging via WebSockets, message status indicators, and date dividers.

TODO: Backend Functionality

A. View active chats and message history

- Chat list topbar
- Click to open conversation
- Scrollable message history
- Load older messages on scroll

B. Real-time messaging between agent and customer

- Send text messages instantly
- Receive messages without refresh
- Connection status indicator

C. Message status indicators (sent, delivered, read)

- Delivered double checkmark
- Read receipt with timestamp
- Add date divider
- Records for the status if conversation is open or closed

D. Attachments (optional: images, documents)

- Image upload (JPG, PNG)
- Document upload (PDF, DOC.)
- File size validation
- File type validation
- Image upload display
- Document upload display

E. Transfer chat to another agent or department (No frontend implemented)

- Transfer to specific agent dropdown
- Transfer to department option

- Transfer reason/note field
- Notification to receiving agent

F. Tag chats by category (e.g., Sales, Support)
- Add multiple tags to conversation
- Remove tags

G. End chat / archive conversation
- End conversation
- Archive to history

D. Quick Responses

Status: Frontend Implemented

Predefined message templates to accelerate response times. Templates are organized into categorized tabs (Sales, Support, General) with expand/collapse functionality.

TODO: Backend Functionality

A. Predefined canned messages (editable by admin)
- Create templates
- Edit existing templates
- Delete templates
- Default templates:

  "Thanks for reaching out! How can I help you today?"

  "Could you provide more details?"

  "Let me transfer you to the right department."

B. One-click insertion into chat
- Click template to auto-fill message input
- Cursor positioning after insert
- Edit before sending option

C. Categorized quick replies (sales, support, general)
- Category tabs: Sales, Support, General
- Expand/collapse categories

E. File Upload System

Status: Frontend Implemented

Implemented (Frontend UI) Integrated into the chat interface, supporting traditional file pickers and drag-and-drop operations with validation for images and documents.

TODO: Backend Functionality
A. Upload media/files

- File picker
- Drag and drop support
- Upload progress bar
- File preview before send
- Cancel upload option
- File size limit display

F. Authentication (Login / Logout)

Status: Only implemented login and logout. The authentication layer is planned to support secure email/password login, session management with timeout handling, and user registration.

TODO: Backend Functionality
A. Login / Logout for Users

- Email and password input fields
- Login button with validation
- Logout functionality
- Session timeout handling
- Sign up

G. Profile Management

Status: Not Implemented
Planned interface for agents and admins to update personal info, department affiliation, and profile photos.

TODO: Backend Functionality
A. Role-based access (Admin, Agent, Viewer)

- Role assignment during user creation
- Admin: Full system access, can assign roles to others
- Agent: Can handle chats, view customer info, use quick responses
- Viewer: Read-only access to chat history and analytics
- Role-based route protection

TODO: Frontend and Backend Functionality
B. Basic Profile management

- Edit name field
- Update email address

- Select/change department
- Upload profile photo
- View profile information

H. Customer Management

Status: Not Implemented

A dedicated module planned for a comprehensive "Customer 360" info panel and internal notes.

TODO: Frontend and Backend Functionality

A. Basic customer info panel (name, email, department)

- Display customer name
- Show customer email
- Indicate department/segment
- Edit customer info (admin only)

B. View the customer's previous chat history

- List of past conversations
- Date and time of each chat
- Quick preview of chat summary
- Click to view full history

C. Tag or add internal notes per customer

- Add private tags to customer
- Write internal notes (agent-only visible)
- Edit/delete notes
- Timestamp on notes

I. Notifications

Status: Not Implemented Planned system for sound and visual alerts for new messages and chat transfers.

Note: Existing Feature Implemented in old project, Lines 51-73:
https://github.com/FORHU/BoostK/blob/main/app/(app)/guest-chat/page.tsx

TODO: Frontend and Backend Functionality

A. New message notifications (sound + visual badge)

- Browser notification permission
- Sound alert toggle

B. Incoming chat alerts

- Popup notification for new chats
- Auto-accept or manual accept option

C. Chat transfer or assignment notifications

- Alert when chat transferred to you
- Show transfer reason
- Accept/decline transfer option
- Previous agent context display

Thanks for reading, enjoy coding with BOOSTK! (ง•̀_•́)ง✧